Lab 03: SOLID Design Principles in Software Construction

Objectives:
After completing this lab, you should be able to:

- Explain the five SOLID design principles.
- Identify SOLID violations in existing Java code.
- Analyze poorly designed classes.
- Refactor code using SRP, OCP, LSP, ISP, and DIP.
- Compare code before and after refactoring.
- Apply SOLID principles to a small software system.

Scenario — Food Delivery System
Throughout this lab, we will use a Food Delivery System.
The system should support:

- Managing orders
- Processing payments
- Sending notifications
- Delivering orders
- Generating receipts

We will first examine poorly designed code and then improve it using the SOLID principles.

Single Responsibility Principle (SRP)
1. What is SRP?
A class should have one responsibility and one reason to change. In simple terms, a class should not handle several unrelated tasks.

Poorly Designed Code:
Consider this class:

```
public class Order {
    public void createOrder() {
        System.out.println("Order created.");
    }
    public void calculateTotal() {
        System.out.println("Calculating order total.");
    }
    public void saveToDatabase() {
        System.out.println("Saving order to database.");
    }
    public void sendEmail() {
        System.out.println("Sending confirmation email.");
    }
    public void printReceipt() {
        System.out.println("Printing receipt.");
    }
}
```

Identify the Problem:
The Order class is responsible for:

- Creating an order
- Calculating the total
- Saving data
- Sending email
- Printing receipts

These responsibilities can change independently. For example:

- Database changes → Order must change.
- Email service changes → Order must change.
- Receipt format changes → Order must change.

This violates SRP.

Refactoring According to SRP
Separate the responsibilities:

```
public class Order {
   public void createOrder() {
      System.out.println("Order created.");
   }
   public void calculateTotal() {
      System.out.println("Calculating order total.");
   }
}
```

Database responsibility:

```
public class OrderRepository {
   public void save(Order order) {
      System.out.println("Saving order to database.");
   }
}
```

Email responsibility:

```
public class EmailService {
   public void sendConfirmation(Order order) {
      System.out.println("Sending confirmation email.");
   }
}
```

Receipt responsibility:

```
public class ReceiptPrinter {
   public void print(Order order) {
      System.out.println("Printing receipt.");
   }
}
```

Result Before:

```
Order
  ├── Order management
  ├── Database
  ├── Email
  └── Receipt
```

After:

- Order
- OrderRepository
- EmailService
- ReceiptPrinter

Each class now has a clear responsibility.

Open/Closed Principle (OCP)

2. What is OCP?

Software entities should be open for extension but closed for modification. This means we should be able to add new behavior without repeatedly modifying existing code.

OCP Violation:

Suppose the food delivery system supports different payment methods.

```
public class PaymentService {
   public void pay(String paymentMethod, double amount) {
      if (paymentMethod.equals("CARD")) {
         System.out.println("Paying using credit card.");
      }
      else if (paymentMethod.equals("PAYPAL")) {
         System.out.println("Paying using PayPal.");
      }
      else if (paymentMethod.equals("CASH")) {
         System.out.println("Paying using cash.");
      }
   }
}
```

Problem: If we introduce:

- JazzCash
- EasyPaisa
- Bank Transfer

we must modify PaymentService. This violates OCP.

Refactoring Using OCP:

Create an abstraction:

```
public interface PaymentMethod {
   void pay(double amount);
}
```

Credit card:

```
public class CreditCardPayment implements PaymentMethod {
   @Override
   public void pay(double amount) {
      System.out.println("Paid " + amount + " using credit card.");
   }
}
```

PayPal:

```
public class PayPalPayment implements PaymentMethod {
   @Override
   public void pay(double amount) {
      System.out.println("Paid " + amount + " using PayPal." );
   }
}
```

Cash:

```
public class CashPayment implements PaymentMethod {
   @Override
   public void pay(double amount) {
      System.out.println("Paid " + amount + " using cash." );
   }
}
```

Payment service:

```
public class PaymentService {
   public void processPayment(
       PaymentMethod paymentMethod,
        double amount) {
      paymentMethod.pay(amount);
   }
}
```

Now a new payment method can be added:

```
public class EasyPaisaPayment implements PaymentMethod {
   @Override
   public void pay(double amount) {
      System.out.println("Paid " + amount + " using EasyPaisa." );
   }
}
```

We did not modify PaymentService. This demonstrates OCP.

Liskov Substitution Principle (LSP)

3. What is LSP?

Objects of a subclass should be usable wherever objects of the parent class are expected without breaking the program.

LSP Violation:

Consider:

```
public class DeliveryVehicle {
   public void deliverOrder() {
      System.out.println("Delivering order.");
   }
}
```

Now create:

```
public class DeliveryBike extends DeliveryVehicle {
   @Override
   public void deliverOrder() {
      System.out.println("Delivering by bike.");
   }
}
```

But suppose we create:

```
public class Drone extends DeliveryVehicle {
   @Override
   public void deliverOrder() {
      throw new UnsupportedOperationException("Drone cannot deliver this type of order." );
   }
}
```

The parent class promises that every DeliveryVehicle can deliver an order, but Drone cannot fulfill that behavior. This creates an LSP violation.

Refactoring the Design:

Define a more appropriate abstraction:

```
public interface Deliverable {
   void deliverOrder();
}
```

Bike:

```
public class DeliveryBike implements Deliverable {
   @Override
   public void deliverOrder() {
      System.out.println("Delivering by bike.");
   }
}
```

Drone:

```
public class Drone implements Deliverable {
   @Override
   public void deliverOrder() {
      System.out.println("Delivering by drone.");
   }}
```

Now every implementation of Deliverable provides the expected behavior.

Identify an LSP Violation:
Consider:

```
public class FoodDelivery {
   public void deliver() {
      System.out.println("Delivering food.");
   }
}
public class RestaurantPickup extends FoodDelivery {
   @Override
   public void deliver() {
      throw new UnsupportedOperationException();
   }
}
```

Identify why this violates LSP.
Create a better abstraction.
Refactor the classes.
Test both restaurant delivery and customer pickup.

Interface Segregation Principle (ISP)
4. What is ISP?
Clients should not be forced to depend on methods they do not use. Instead of creating one large interface, create smaller interfaces based on specific responsibilities.

ISP Violation:

```
public interface RestaurantService {
   void prepareFood();
   void deliverFood();
   void processPayment();
   void printReceipt();
}
```

Now suppose a Kitchen only prepares food.

```
public class Kitchen implements RestaurantService {
   @Override
   public void prepareFood() {
      System.out.println("Preparing food.");     }
   @Override
   public void deliverFood() {
      throw new UnsupportedOperationException();    }
   @Override
   public void processPayment() {
      throw new UnsupportedOperationException();    }
   @Override
   public void printReceipt() {
      throw new UnsupportedOperationException();    }
}
```

The Kitchen is forced to implement methods it does not need. This violates ISP.

Refactoring Using ISP:
Create smaller interfaces:

```
public interface FoodPreparation {
   void prepareFood();
}
public interface FoodDelivery {
   void deliverFood();
}
public interface PaymentProcessing {
   void processPayment();
}
```

Now:

```
public class Kitchen implements FoodPreparation {
   @Override
   public void prepareFood() {
      System.out.println("Preparing food.");
   }
}
```

Delivery service:

```
public class DeliveryService implements FoodDelivery {
   @Override
   public void deliverFood() {
      System.out.println("Delivering food.");
   }
}
```

Payment service:

```
public class PaymentService
      implements PaymentProcessing {
   @Override
   public void processPayment() {
      System.out.println("Processing payment.");
   }
}
```

Result:
Instead of:

```
Large Interface
      ↓
   Kitchen
   Delivery
   Payment
```

we have:

- FoodPreparation → Kitchen
- FoodDelivery → DeliveryService
- PaymentProcessing → PaymentService

Dependency Inversion Principle (DIP)

5. What is DIP?

High-level modules should not depend directly on low-level modules. Both should depend on abstractions.

DIP Violation:

```
public class OrderService {
   private MySQLOrderRepository repository;
   public OrderService() {
      repository = new MySQLOrderRepository();
   }
   public void saveOrder(Order order) {
      repository.save(order);
   }
}
```

Problem here is "OrderService" directly depends on: MySQLOrderRepository

If the system changes from MySQL to MongoDB, OrderService must be modified.

Refactoring Using DIP

Create an abstraction:

```
public interface OrderRepository {
   void save(Order order);
}
```

MySQL implementation:

```
public class MySQLOrderRepository
      implements OrderRepository {
   @Override
   public void save(Order order) {
      System.out.println("Saving order in MySQL.");
   }
}
```

MongoDB implementation:

```
public class MongoOrderRepository
      implements OrderRepository {
   @Override
   public void save(Order order) {
      System.out.println("Saving order in MongoDB.");
   }
}
```

Now modify **OrderService**:

```
public class OrderService {
   private final OrderRepository repository;
   public OrderService(OrderRepository repository) {
      this.repository = repository;
   }
   public void saveOrder(Order order) {
      repository.save(order);
   }
}
```

Usage:

```
OrderRepository repository = new MySQLOrderRepository();
OrderService service = new OrderService(repository);
```

The same service can now work with MongoDB:

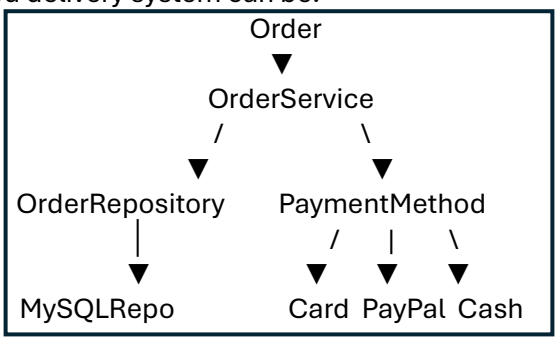
```
OrderRepository repository = new MongoOrderRepository();
OrderService service = new OrderService(repository);
```

No modification to OrderService is required.

Putting SOLID Together

19. Integrated Design

A simplified design for the food delivery system can be:



- FoodPreparation → Kitchen
- FoodDelivery → DeliveryService
- NotificationService → Email / SMS

Refactoring Exercise:

Poorly Designed Food Delivery Class

Create the following class:

```
public class FoodDeliverySystem {
    public void createOrder() {
        System.out.println("Creating order.");    }
    public void saveOrderToMySQL() {
        System.out.println("Saving order to MySQL.");   }
    public void payByCard(double amount) {
        System.out.println("Paying by card: " + amount);    }
    public void payByPayPal(double amount) {
        System.out.println("Paying by PayPal: " + amount);    }
    public void sendEmail() {
        System.out.println("Sending email.");     }
    public void sendSMS() {
        System.out.println("Sending SMS.");       }
    public void prepareFood() {
        System.out.println("Preparing food.");      }
    public void deliverFood() {
        System.out.println("Delivering food.");     }
}
```

Identify the Problems
Find at least five design problems in this class.
Consider:
How many responsibilities does the class have?
What happens when another payment method is added?
What happens when the database changes?
Which methods could be grouped into smaller interfaces?
Which parts should depend on abstractions?

Refactor the above class by creating at least the following:

- Order
- OrderRepository
- MySQLOrderRepository
- PaymentMethod
- CreditCardPayment
- PayPalPayment
- FoodPreparation
- Kitchen
- FoodDelivery
- DeliveryService
- NotificationService
- EmailNotification
- SMSNotification
- OrderService

Your final design must:

- Apply SRP by separating responsibilities.
- Apply OCP by allowing new payment methods.
- Apply LSP by ensuring implementations can replace their abstractions.
- Apply ISP by using focused interfaces.
- Apply DIP by injecting dependencies into high-level classes.

After Refactoring
Order.java

```
public class Order {
   private final int orderId;
   private final String customerName;
   private final double amount;
   public Order(int orderId, String customerName, double amount) {
      this.orderId = orderId;
      this.customerName = customerName;
      this.amount = amount; }
   public int getOrderId() {
      return orderId; }
   public String getCustomerName() {
      return customerName; }
   public double getAmount() {
      return amount;   } }
```

OrderRepository.java

```
public interface OrderRepository {
  void save(Order order);  }
```

MySQLOrderRepository.java

```
public class MySQLOrderRepository implements OrderRepository {
  @Override
  public void save(Order order) {
    System.out.println(
      "Order " + order.getOrderId() +
      " saved to MySQL database." );  } }
```

PaymentMethod.java

```
public interface PaymentMethod {
  void pay(double amount); }
```

CreditCardPayment.java

```
public class CreditCardPayment implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println(
      "Paid " + amount + " using credit card." );  } }
```

PayPalPayment.java

```
public class PayPalPayment implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println(
      "Paid " + amount + " using PayPal." );  }  }
```

FoodPreparation.java

```
public interface FoodPreparation {
  void prepareFood();  }
```

Kitchen.java

```
public class Kitchen implements FoodPreparation {
  @Override
  public void prepareFood() {
    System.out.println("Preparing food in the kitchen.");  }  }
```

FoodDelivery.java

```
public interface FoodDelivery {
  void deliverFood();  }
```

DeliveryService.java

```
public class DeliveryService implements FoodDelivery {
  @Override
  public void deliverFood() {
    System.out.println("Delivering food to the customer.");  }  }
```

NotificationService.java

```
public interface NotificationService {
  void sendNotification(String message);  }
```

EmailNotification.java

```
public class EmailNotification
    implements NotificationService {
  @Override
  public void sendNotification(String message) {
    System.out.println("Email notification: " + message );  } }
```

SMSNotification.java

```
public class SMSNotification
    implements NotificationService {
  @Override
  public void sendNotification(String message) {
    System.out.println("SMS notification: " + message );   }  }
```

OrderService.java

This is the **high-level class**. It does not directly create or depend on MySQL, a specific payment method, or a specific notification system.

```
  public class OrderService {
    private final OrderRepository orderRepository;
    private final PaymentMethod paymentMethod;
    private final FoodPreparation foodPreparation;
    private final FoodDelivery foodDelivery;
    private final NotificationService notificationService;
    public OrderService(
        OrderRepository orderRepository,
        PaymentMethod paymentMethod,
        FoodPreparation foodPreparation,
        FoodDelivery foodDelivery,
        NotificationService notificationService) {
      this.orderRepository = orderRepository;
      this.paymentMethod = paymentMethod;
      this.foodPreparation = foodPreparation;
      this.foodDelivery = foodDelivery;
      this.notificationService = notificationService;
    }
    public void placeOrder(Order order) {
      System.out.println("Placing order...");
      orderRepository.save(order);
      paymentMethod.pay(order.getAmount());
      foodPreparation.prepareFood();
      foodDelivery.deliverFood();
      notificationService.sendNotification(
        "Order " + order.getOrderId() +
        " has been delivered."
  );
  }
}
```

Main.java

```
public class Main {
   public static void main(String[] args) {
      Order order = new Order(101, "Ali", 2500  );
      OrderRepository repository = new MySQLOrderRepository();
      PaymentMethod payment = new CreditCardPayment();
      FoodPreparation kitchen = new Kitchen();
      FoodDelivery delivery =  new DeliveryService();
      NotificationService notification = new EmailNotification();
      OrderService orderService = new OrderService(
            repository,
            payment,
            kitchen,
            delivery,
            notification );
      orderService.placeOrder(order);
   }
}
```

Example: Adding a New Payment Method

You can add another payment method without changing OrderService:

```
public class CashPayment implements PaymentMethod {
   @Override
   public void pay(double amount) {
      System.out.println(
         "Paid " + amount + " using cash."
      );
   }
}
```

Then simply use:

```
PaymentMethod payment = new CashPayment();
```

The OrderService remains unchanged. This is the practical benefit of **OCP + DIP**.